

BiteBit: An Explicitly Typed Minimal
Language for Bitwise and Low-Level
Computation
[Language Specification]

1. Introduction.....	3
2. Lexical Structure.....	3
3. Syntax.....	4
4. Semantics.....	5
5. Example Program.....	7

1. Introduction

1.1 Purpose

This document defines the syntax and semantics of the BiteBit programming language. It is meant for users of the language and for anyone building tools for BiteBit, such as a tokenizer, parser, or interpreter.

1.2 Scope

BiteBit is a small typed language made for bitwise operations, binary and hexadecimal values, and simple low-level numeric programming. This specification covers the basic parts of the language, including tokens, grammar, types, variables, expressions, control flow, and functions.

2. Lexical Structure

2.1 Character Set

A BiteBit program is a sequence of Unicode characters. Whitespace is used to separate tokens and is otherwise ignored, except inside string literals.

2.2 Comments

Comments begin with `//` and continue to the end of the line.

Example:

```
// comments
```

2.3 Identifiers

An identifier begins with a letter or underscore and may contain letters, digits, and underscores after that.

Valid examples

`x`

`_mask`

`count1`

Invalid examples`labc``my-var`**2.4 Keywords**

The following words are reserved in BiteBit:

`let``fn``return``if``else``while``true``false``null`**2.5 Data Types**

BiteBit uses these built-in data types:

`Int``Bool``String``Null`**2.6 Literals**

BiteBit supports:

- decimal integers like `10`
- binary integers like `0b1010`
- hexadecimal integers like `0x2A`
- boolean literals `true` and `false`
- string literals like `"hello"`

- the null literal `null`

3. Syntax

The syntax of BiteBit is described using EBNF.

```
program = { declaration | statement } ;
```

```
declaration = var_decl | func_decl ;
```

```
var_decl = "let" identifier ":" type "=" expression ";" ;
```

```
func_decl = "fn" identifier "(" [ params ] ")" ":" type block ;
```

```
params = param { "," param } ;
```

```
param = identifier ":" type ;
```

```
type = "Int" | "Bool" | "String" | "Null" ;
```

```
statement = assignment
```

```
          | if_stmt
```

```
          | while_stmt
```

```
          | return_stmt
```

```
          | expr_stmt
```

```
          | block ;
```

```
assignment = identifier "=" expression ";" ;
```

```
if_stmt = "if" "(" expression ")" block [ "else" block ] ;
```

```
while_stmt = "while" "(" expression ")" block ;
```

```
return_stmt = "return" [ expression ] ";" ;
```

```
expr_stmt = expression ";" ;
```

```
block = "{" { declaration | statement } "}" ;
```

```
expression = equality ;
```

```
equality = comparison { ("==" | "!=") comparison } ;
```

```

comparison = bitwise_or { ("<" | "<=" | ">" | ">=") bitwise_or }
;
bitwise_or = bitwise_xor { "|" bitwise_xor } ;
bitwise_xor = bitwise_and { "^" bitwise_and } ;
bitwise_and = shift { "&" shift } ;
shift = term { ("<<" | ">>") term } ;
term = factor { ("+" | "-") factor } ;
factor = unary { ("*" | "/" | "%") unary } ;
unary = ("!" | "-" | "~") unary | primary ;
primary = literal | identifier | call_expr | "(" expression ")"
;
call_expr = identifier "(" [ args ] ")" ;
args = expression { "," expression } ;

```

4. Semantics

4.1 Data Types

BiteBit has four built-in types:

```

Int
Bool
String
Null

```

4.2 Variables

Variables are declared using `let` and must include a type.

Examples

```

let x : Int = 10;
let name : String = "Sam";
let flag : Bool = true;

```

4.3 Expressions

Expressions evaluate to a value.

BiteBit supports:

- arithmetic operators: `+` `-` `*` `/` `%`
- comparison operators: `==` `!=` `<` `<=` `>` `>=`
- bitwise operators: `&` `|` `^` `~` `<<` `>>`
- logical negation: `!`

4.4 Type Rules

BiteBit is explicitly typed. A variable's declared type must match the value assigned to it. The preliminary symbol table also checks this and prints errors for invalid assignments and declarations.

Valid example

```
let x : Int = 10;
let ok : Bool = true;
let label : String = "bit";
```

Invalid example

```
let x : Int = "hello";
```

4.5 Control Flow

BiteBit supports:

- `if` and `else`
- `while`

Example

```
if (flag) {
    x = x << 1;
}
```

4.6 Functions

Functions are declared using `fn`, followed by a name, parameter list, return type, and block.

Example

```
fn lowByte(x : Int) : Int {  
    return x & 0xFF;  
}
```

4.7 Scope

Variables are block-scoped. A variable declared inside a block exists only inside that block.

5. Example Program

```
let x : Int = 0xABCD;  
let low : Int = x & 0xFF;  
let moved : Int = low << 8;
```

This example shows the main purpose of BiteBit because it uses hexadecimal values, masking, and shifts, which are central to the language.